



UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA

Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

*Laboratorio No. 8 · Control del rol y prevención de escalada de privilegios ·
API_TEATRO*

ESPACIO ACADÉMICO:

Seguridad en el Desarrollo de Software

PRESENTADO A:

Sergio Arley Puerto Moreno

AUTORES:

Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. El problema

Al terminar el Laboratorio 7 dejamos una puerta abierta y la anotamos como pendiente: cualquiera con una API Key válida podía registrarse como administrador. Bastaba con mandar el campo `rol` en el cuerpo de la petición a `POST /api/auth/registro`:

```
{ "nombre": "Usuario Ataque", "email": "ataque@teatro.com",  
  "password": "ClaveSegura2026!", "rol": "administrador" }
```

Así se veía el contrato publicado en Swagger antes de este laboratorio. El `rol` aparecía como campo obligatorio del registro, o sea que la propia documentación invitaba al cliente a elegirlo:

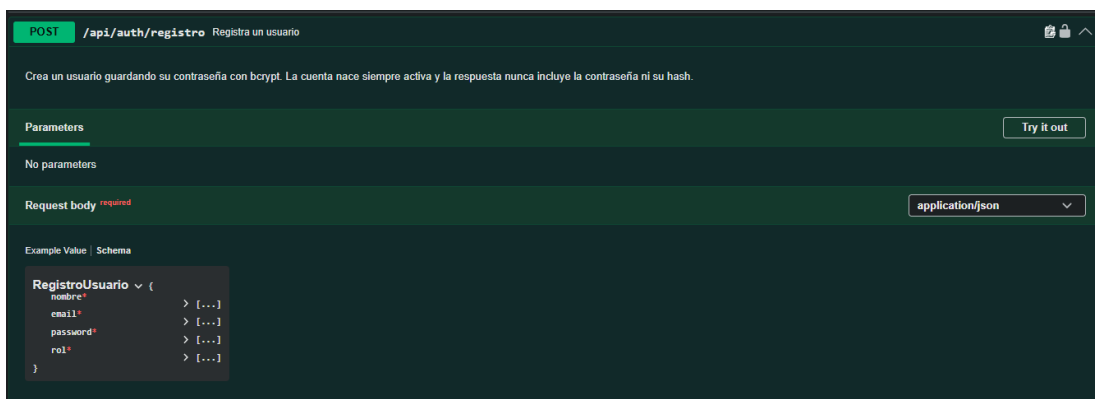


Figura 1. Contrato del registro antes del Laboratorio 8: `RegistroUsuario` pedía `rol` como campo obligatorio.

Hoy el daño es cero, porque el `rol` todavía no da acceso a nada. Pero el bloque siguiente va a hacer que el `rol` decida permisos, y ese día esta puerta pasa a ser grave. La cerramos antes de que importe.

En el fallo colaboraban dos capas:

- El validador declaraba `body("rol").isIn(ROLES)`, así que el `rol` era un campo válido del contrato y `matchedData` lo dejaba pasar.
- El service hacía `rol: datos.rol`, o sea que se fiaba de lo que llegaba desde el cliente.

Cliente controla el `rol` + service confía en el `rol` = escalada de privilegios. Por eso arreglamos las dos capas y no solo una. Si solo quitáramos la validación, el service seguiría leyendo `datos.rol` y guardaría `undefined`. Si solo arregláramos el service, el `rol` seguiría siendo un campo aceptado del contrato. Cada capa tiene que ser correcta por su cuenta.

2. El contrato público en Swagger

En `src/routes/auth.routes.js`, el schema `RegistroUsuario` perdió la propiedad `rol`, y `rol` salió de la lista `required`. Ahora quedan tres propiedades obligatorias: `nombre`, `email` y `password`.

Además, la descripción de `POST /api/auth/registro` dice ahora con todas las letras que el rol lo asigna el servidor y que cualquier rol enviado por el cliente se ignora. Si el schema siguiera anunciando `rol`, estaríamos documentando una entrada que el servidor ya descarta. Así Swagger sirve también como documentación de la seguridad: quien lo lea sabe, sin mirar el código, que ese campo no le pertenece al cliente.

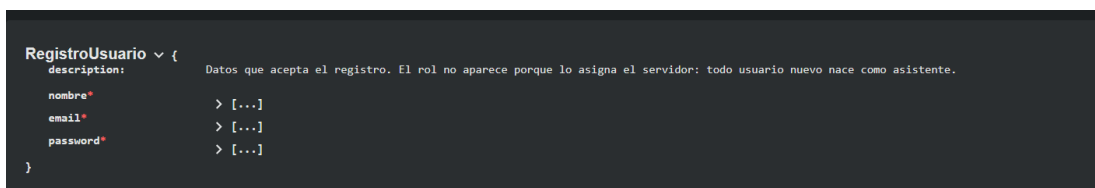


Figura 2. Contrato del registro después del Laboratorio 8: `RegistroUsuario` pide solo nombre, email y password, y su descripción indica que el rol lo asigna el servidor.

`LoginUsuario`, `UsuarioPublico` y `RespuestaUsuario` no cambiaron. `UsuarioPublico` sigue devolviendo `rol`, porque el rol sí sale en la respuesta aunque no entre en la petición: lo decide el servidor.

3. Primera defensa: el validador

En `src/middlewares/auth.validator.js` quitamos entero el bloque de `body("rol")`. `validarRegistro` quedó con tres reglas: nombre, email y password. `validarLogin` no se tocó.

Al quitarlo, la importación de `ROLES` quedaba sin usar y también la eliminamos, para no dejar un `require` muerto. `ROLES` sigue viviendo en `usuarios.service.js` y sigue exportándose: ya no sirve para validar la entrada, pero es la lista de roles del sistema y el bloque siguiente la necesitará para decidir permisos.

```

12 const reglaNombre = () =>
18   .withMessage("El nombre debe tener entre 3 y 100 caracteres");
19
20 const reglaEmail = () =>
21   body("email")
22     .isEmail()
23     .withMessage("Debe proporcionar un correo electrónico válido")
24     .normalizeEmail();
25
26 // bcrypt solo procesa 72 bytes de entrada, así que lo que pase de ahí se
27 // ignoraría en silencio. Se rechaza antes en vez de aceptarlo a medias.
28 const reglaPassword = () =>
29   body("password")
30     .isString()
31     .withMessage("La contraseña debe ser texto")
32     .length({ min: 10, max: 72 })
33     .withMessage("La contraseña debe tener entre 10 y 72 caracteres");
34
35 // =====
36 // Registro
37 // El rol no se declara: lo asigna el servidor, no el cliente
38 // =====
39 const validarRegistro = [reglaNombre(), reglaEmail(), reglaPassword()];
40

```

Figura 3. Primera defensa en `auth.validator.js`: `validarRegistro` ya no declara rol, así que `matchedData` lo descarta.

4. Segunda defensa: el service

En `src/services/usuarios.service.js` la línea `rol: datos.rol` pasó a ser un rol fijo controlado por el servidor. En el teatro es `asistente`, el menos privilegiado de los tres: el que compra boletas, equivalente a `paciente` en el hospital del laboratorio de referencia. Lo definimos como constante junto a `ROLES`:

```

const ROLES = ["administrador", "taquilla", "asistente"];
const ROL_POR_DEFECTO = "asistente";

```

Y dentro de `crearUsuario`, junto a `activo: true`, que ya estaba forzado desde el Laboratorio 7, fijamos el rol:

```

// Valores controlados por el servidor
rol: ROL_POR_DEFECTO,
activo: true

```

El usuario se sigue construyendo campo por campo con un objeto literal, nunca con `...datos`. Eso es allowlisting explícito: solo entra lo que nombramos.

```
src > services > usuarios.service.js > [unknown]
1 // =====
2 // Importaciones
3 // =====
4 const usuarios = require("../data/usuarios");
5 const {
6   generarPasswordHash,
7   verificarPassword
8 } = require("../utils/password.util");
9
10 // =====
11 // Roles del sistema
12 // Todo registro nace con el rol menos privilegiado
13 // =====
14 const ROLES = ["administrador", "taquilla", "asistente"];
15 const ROL_POR_DEFECTO = "asistente";
16
17 // =====
18 // Obtener usuario por email
19 // El email es único, comparado siempre en minúsculas
20 // =====
21 const obtenerUsuarioPorEmail = (email) =>
22   usuarios.find(
23     (usuario) => usuario.email.toLowerCase() === email.toLowerCase()
24   );
25
26 // =====
27
28 // =====
29
30 // =====
31
32 // =====
33
34 // =====
35 // El objeto se arma campo por campo, nunca con spread de datos: solo entra lo
36 // que se nombra aquí.
37 // =====
38 const crearUsuario = async (datos) => {
39   const passwordHash = await generarPasswordHash(datos.password);
40
41   const nuevoUsuario = {
42     id:
43       usuarios.length > 0
44       ? Math.max(...usuarios.map((usuario) => usuario.id)) + 1
45       : 1,
46     nombre: datos.nombre,
47     email: datos.email.toLowerCase(),
48     passwordHash,
49     // =====
50     // Valores controlados por el servidor
51     // El cliente no puede fijar su rol ni activarse a sí mismo
52     // =====
53     rol: ROL_POR_DEFECTO,
54     activo: true
55   };
56
57   usuarios.push(nuevoUsuario);
58
59   return nuevoUsuario;
60 };
```

Figura 4. Segunda defensa en usuarios.service.js: la constante ROL_POR_DEFECTO y su uso en crearUsuario, junto a activo: true.

El controller no se tocó

src/controllers/auth.controller.js no cambió ni una línea, y eso es buena señal. Ya leía el cuerpo con matchedData(req, { locations: ["body"] }) y ya le pasaba los datos limpios al service. Cada capa responde su propia pregunta:

Capa	Pregunta que resuelve
Validador (auth.validator.js)	¿Qué datos acepto del cliente?
Controller (auth.controller.js)	¿Qué operación realizo?
Service (usuarios.service.js)	¿Cómo construyo y guardo el usuario?

5. Por qué responde 201 y no 400

Mandar "rol": "administrador" en el registro no devuelve un error 400: devuelve 201 Created, y el usuario queda registrado como asistente. No es un descuido. La entrada se filtra por lista blanca (allowlisting): el validador declara los campos que aceptamos y

`matchedData` extrae únicamente esos. Todo lo demás se descarta, así que la petición es válida y se procesa con los valores del servidor.

Una lista negra tendría que enumerar cada campo peligroso uno por uno, y siempre se queda corta: basta olvidar uno. La lista blanca frena también los campos que nadie previó. Enviar un atributo no significa tener autorización para controlarlo.

6. Atacando nuestra propia API

Con el servidor recién iniciado, hicimos las pruebas en Postman contra `/api/auth/registro` y `/api/auth/login`, enviando la cabecera `X-API-Key` de un cliente activo.

Registro normal sin enviar rol

Enviamos solo nombre, email y password para Asistente Teatro. El cliente mandó tres campos y en la respuesta `201 Created` salen cinco: `rol: "asistente"` y `activo: true` los puso el servidor sin que nadie se los pidiera.

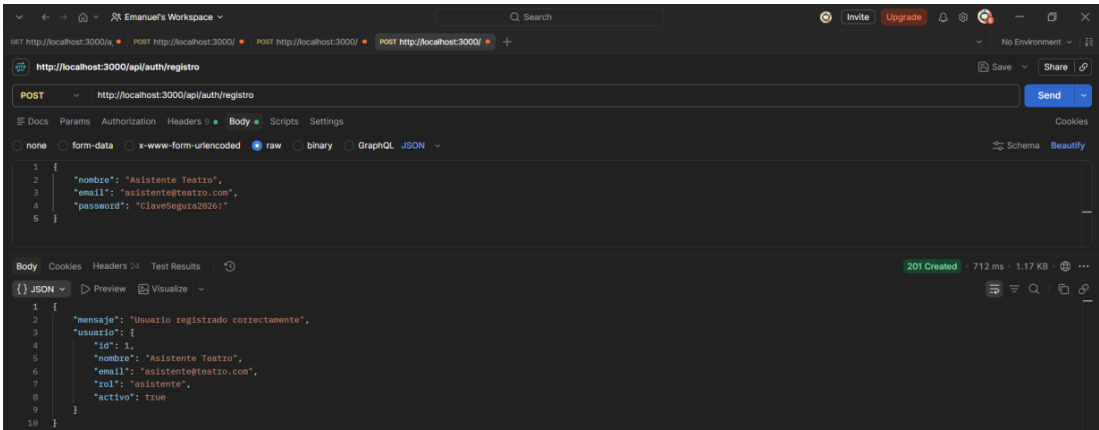


Figura 5. Registro normal: el cliente envía tres campos y el servidor asigna rol "asistente" y activo: true.

Intento de escalada de privilegios

En la segunda petición enviamos `"rol": "administrador"`, `"activo": false` y `"esSuperAdmin": true`. El servidor respondió `201 Created`, pero ignoró los tres atributos. Un `201` no significa que el ataque funcionó: significa que el usuario se creó. Lo que importa es qué quedó guardado:

El atacante pidió	El servidor guardó
<code>rol: administrador</code>	<code>rol: asistente</code>
<code>activo: false</code>	<code>activo: true</code>
<code>esSuperAdmin: true</code>	Ni siquiera existe

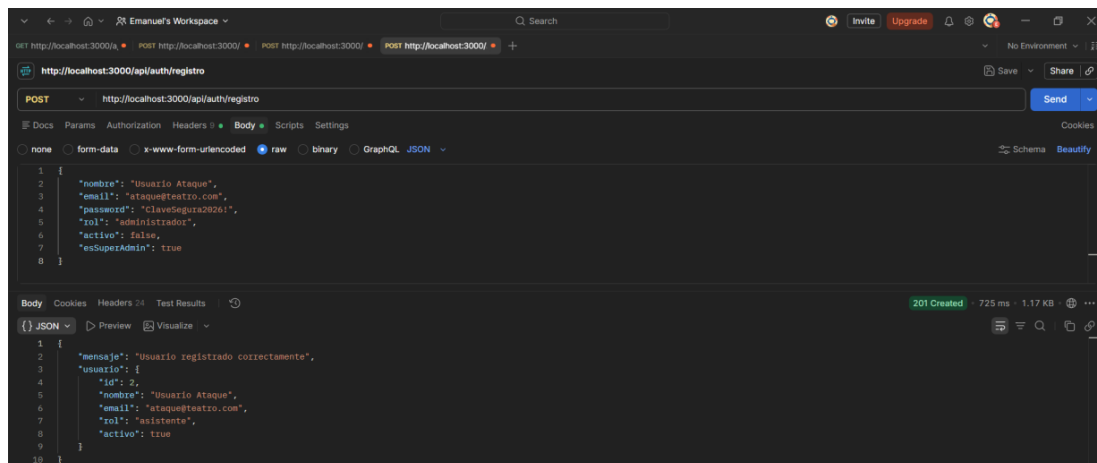


Figura 6. Intento de escalada pidiendo rol "administrador", activo: false y esSuperAdmin: true. El servidor guarda rol "asistente" y activo: true.

Mass assignment completo

En la tercera petición mandamos todos los campos internos y ficticios a la vez: `id: 9999`, `rol: "administrador"`, `activo: false`, `passwordHash: "HASH_CONTROLADO"`, `esSuperAdmin: true` y `permisos: ["DELETE_ALL", "ADMIN"]`. El usuario quedó con `id: 3` y no 9999, `rol: "asistente"` y `activo: true`, sin rastro de los demás.

El campo `permisos` no aparece en ninguna regla ni en ninguna lista de prohibidos del proyecto. Se cayó solo, porque `matchedData` trabaja por lista blanca.

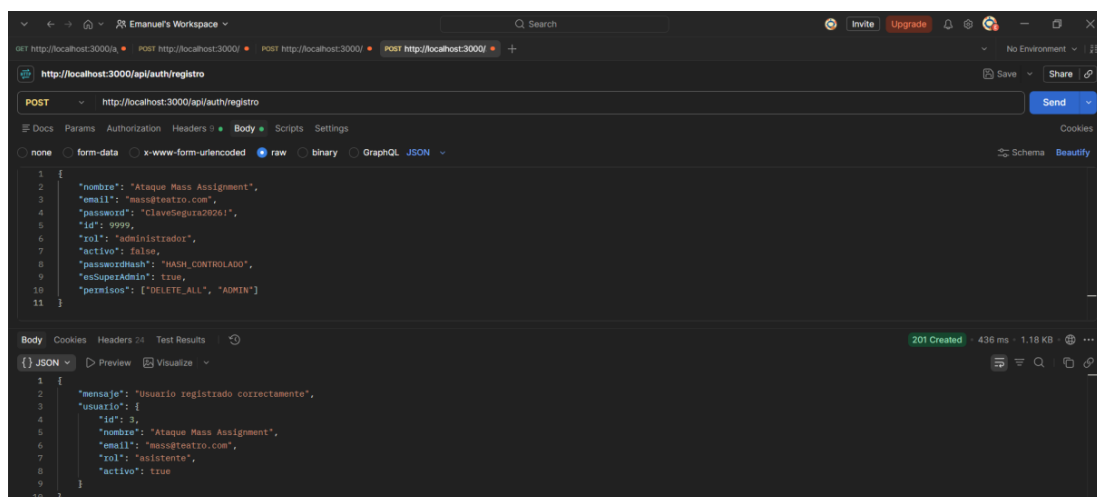


Figura 7. Mass assignment completo: `id`, `rol`, `activo`, `passwordHash`, `esSuperAdmin` y `permisos` se descartan como entrada.

Comprobación con el inicio de sesión

Para demostrar que `passwordHash: "HASH_CONTROLADO"` nunca entró, iniciamos sesión con `mass@teatro.com` y su contraseña real. El login respondió `200 OK` con `rol: "asistente"`. Si el hash falso se hubiera guardado, `bcrypt.compare` habría fallado siempre, con cualquier contraseña.

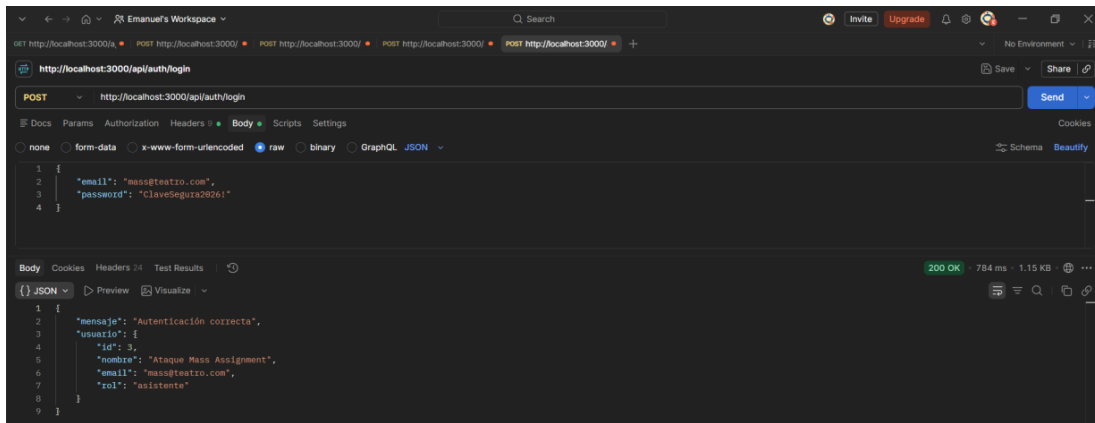


Figura 8. Login correcto con mass@teatro.com: bcrypt generó su propio hash e ignoró el passwordHash falso del registro.

Con el mismo correo y una contraseña equivocada la respuesta es 401 Unauthorized con el mensaje genérico "Credenciales inválidas", que no dice si falló el correo o la contraseña.

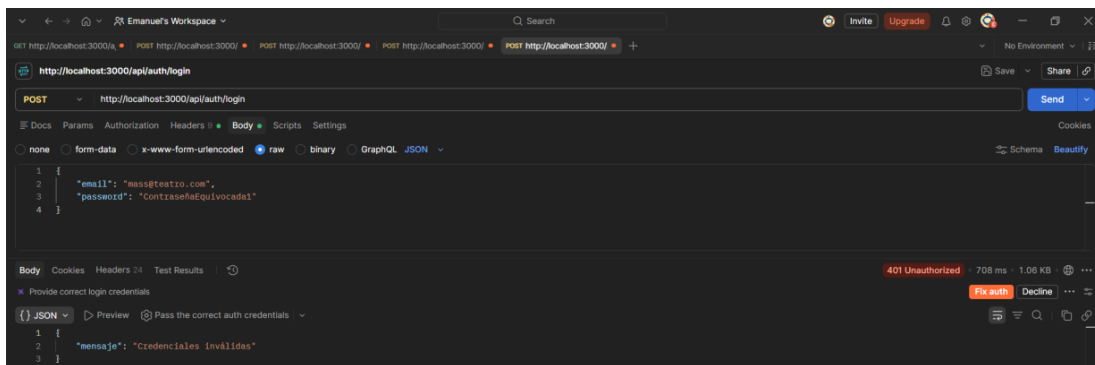


Figura 9. Login con contraseña equivocada: 401 con el mensaje genérico "Credenciales inválidas".

7. Checkpoint del Bloque 4B

Estas son las siete pruebas que pide el laboratorio para dar por cerrado el bloque:

Prueba	Esperado	Obtenido	Estado
Registro sin rol	201 + rol "asistente"	201 + rol "asistente"	OK
Registro con rol "administrador"	201 + rol "asistente"	201 + rol "asistente"	OK
Enviar activo: false	Se mantiene true	activo: true	OK
Enviar passwordHash falso	El campo se ignora	Ignorado; el login con la contraseña real da 200	OK
Enviar esSuperAdmin: true	El campo se ignora	No existe en el usuario	OK
Login con credenciales correctas	200 OK	200 OK	OK
Login con credenciales incorrectas	401 Unauthorized	401 "Credenciales inválidas"	OK

8. ¿De dónde sale el primer administrador?

Ahora todo usuario que entra por `POST /api/auth/registro` nace como asistente. Eso deja una pregunta abierta: si nadie puede registrarse como administrador, ¿cómo se crea el primero?

No vamos a volver a abrir el rol en el registro público. Cuando la API tenga base de datos, el primer administrador se creará con un proceso controlado del lado del servidor, como una inicialización (seed) o una migración administrativa, que no depende de ninguna petición del cliente.

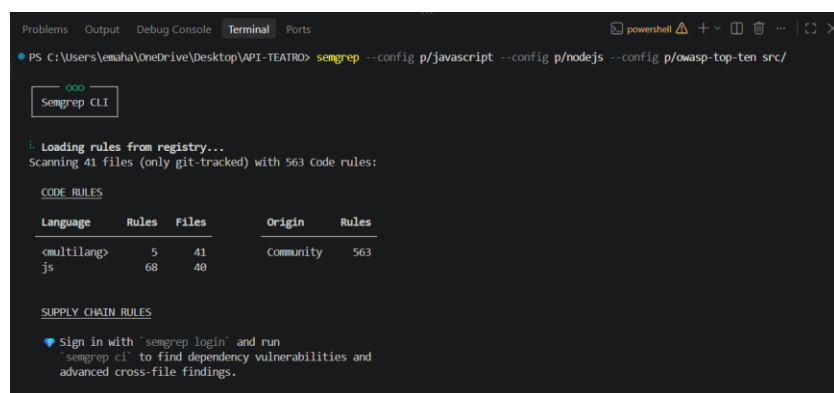
Y cuando existan JWT y autorización, crear usuarios con otro rol, por ejemplo una persona de la taquilla, será una operación aparte que solo podrá ejecutar un administrador ya autenticado. Cualquier otro recibirá 403.

9. Verificación con Sengrep y npm audit

Al terminar volvimos a ejecutar las herramientas SAST y SCA sobre el proyecto:

Herramienta	Lab 7	Lab 8
Sengrep: archivos escaneados (73 reglas)	41	41
Sengrep: hallazgos	0	0
npm audit (146 paquetes)	0 vulnerabilidades	0 vulnerabilidades

Los números cuadran porque en este laboratorio no creamos archivos ni instalamos dependencias: solo modificamos `auth.validator.js`, `usuarios.service.js` y `auth.routes.js`. La API mantiene sus 41 endpoints en 21 rutas, 34 schemas y 7 tags, y las 224 pruebas automatizadas siguen pasando.



```
PS C:\Users\emaha\OneDrive\Desktop\API-TEATRO> sengrep --config p/javascript --config p/nodejs --config p/owasp-top-ten src/

Sengrep CLI

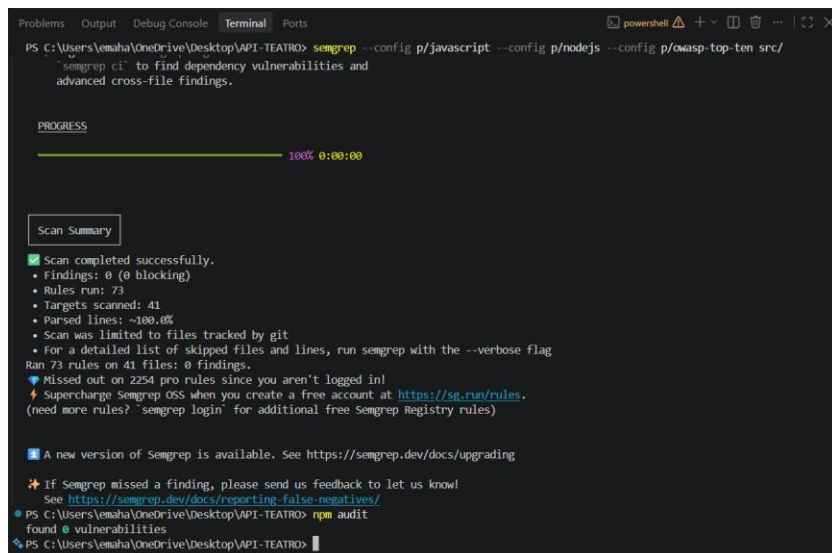
Loading rules from registry...
Scanning 41 files (only git-tracked) with 563 Code rules:

CODE RULES
+-----+-----+-----+-----+-----+
| Language | Rules | Files | Origin | Rules |
+-----+-----+-----+-----+-----+
| <multilang> | 5 | 41 | Community | 563 |
| js | 68 | 40 | | |
+-----+-----+-----+-----+-----+

SUPPLY CHAIN RULES

Sign in with 'sengrep login' and run
'sengrep ci' to find dependency vulnerabilities and
advanced cross-file findings.
```

Figura 10. Sengrep sobre src/: 41 archivos y 73 reglas aplicables (5 multilenguaje y 68 de JavaScript).



```
PS C:\Users\emaha\OneDrive\Desktop\API-TEATRO> semgrep --config p/javascript --config p/nodejs --config p/owasp-top-ten src/
'semgrep ci' to find dependency vulnerabilities and
advanced cross-file findings.

PROGRESS
100% 0:00:00

Scan Summary
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 73
• Targets scanned: 41
• Parsed lines: ~100.0%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 73 rules on 41 files: 0 findings.
💡 Missed out on 2254 pro rules since you aren't logged in!
🔥 Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

📢 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

🌟 If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
PS C:\Users\emaha\OneDrive\Desktop\API-TEATRO> npm audit
found 0 vulnerabilities
PS C:\Users\emaha\OneDrive\Desktop\API-TEATRO>
```

Figura 11. Resumen de Semgrep con 0 hallazgos, seguido de npm audit con 0 vulnerabilidades.

10. Lo que este laboratorio no resuelve

- Por el registro público ya nadie puede nacer administrador. Pero eso también significa que hoy no hay ninguna forma de crear un administrador ni un usuario de taquilla: falta la operación administrativa, y para eso hacen falta JWT y control de rol.
- El rol sigue sin dar ni quitar acceso. Un asistente autenticado puede llamar a los mismos endpoints que un administrador. Este laboratorio arregló quién decide el rol; lo que el rol permite hacer llega en el siguiente bloque.
- Los usuarios siguen en memoria y desaparecen cada vez que se reinicia el servidor.

11. Repositorio

<https://github.com/JuanK2550/API-TEATRO>

El control del rol y la prevención de escalada de privilegios están en el commit [6d78b73](#), y las salidas de Semgrep y npm audit del Laboratorio 8 en [c686a8c](#).